

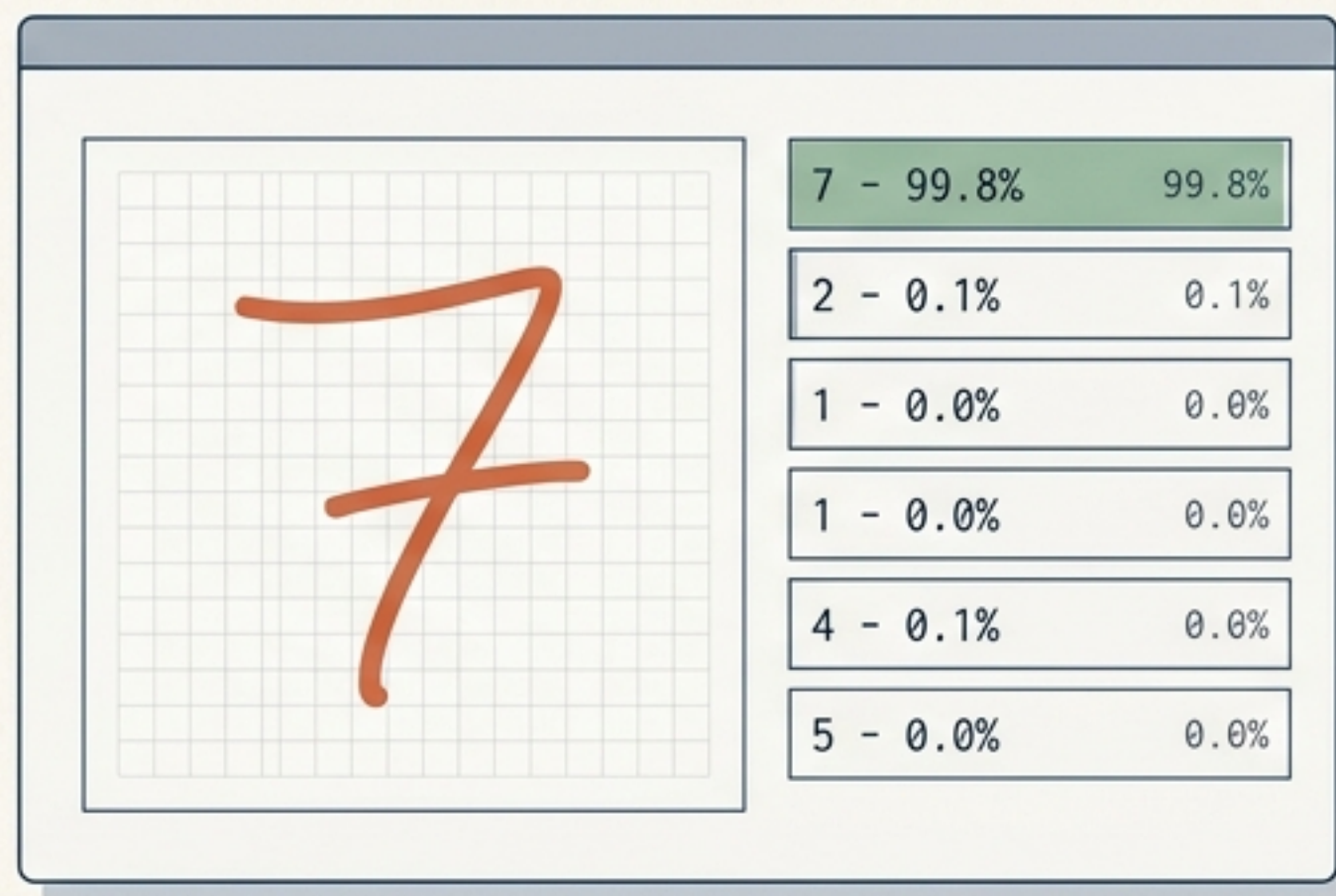
构建全栈 AI 应用：CNN 手写数字识别

教学最小闭环：从模型训练到 Web 画板无服务器部署

Raw MNIST Input Data (28x28 Matrix)

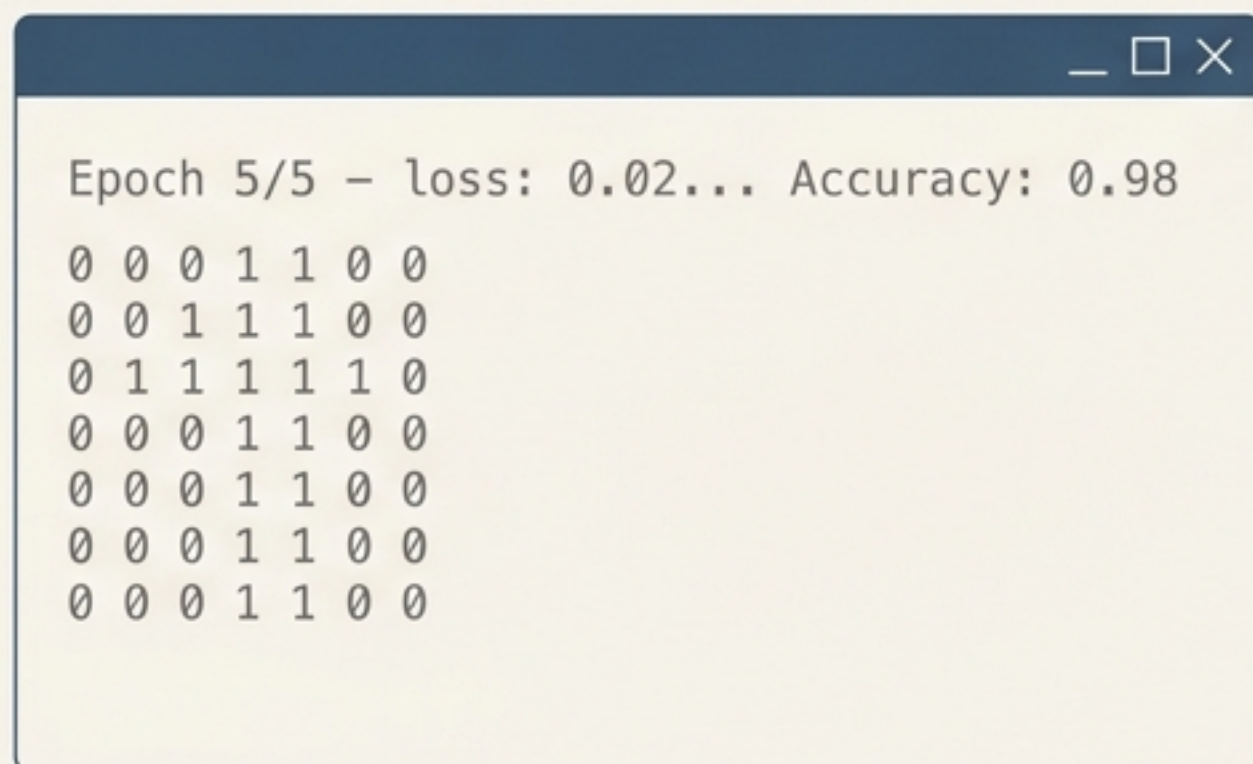


Web Interface & Model Prediction



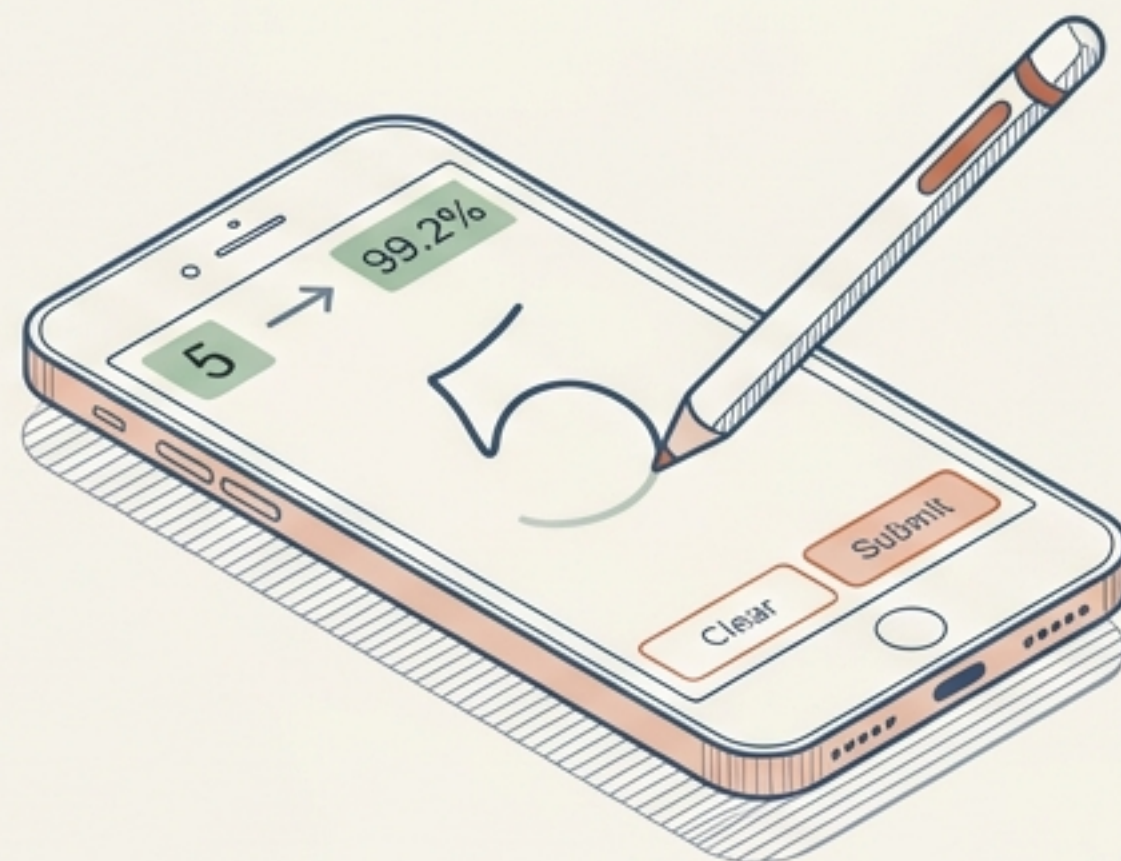
课堂展示的进化：告别控制台，打造真实产品交互

传统方式（脚本级演示）



抽象、枯燥、缺乏互动感。

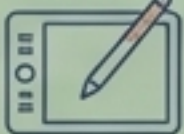
我们的目标（产品级体验）



学生直接在网页写数字，模型实时识别，
体验真实的 AI 产品交互。

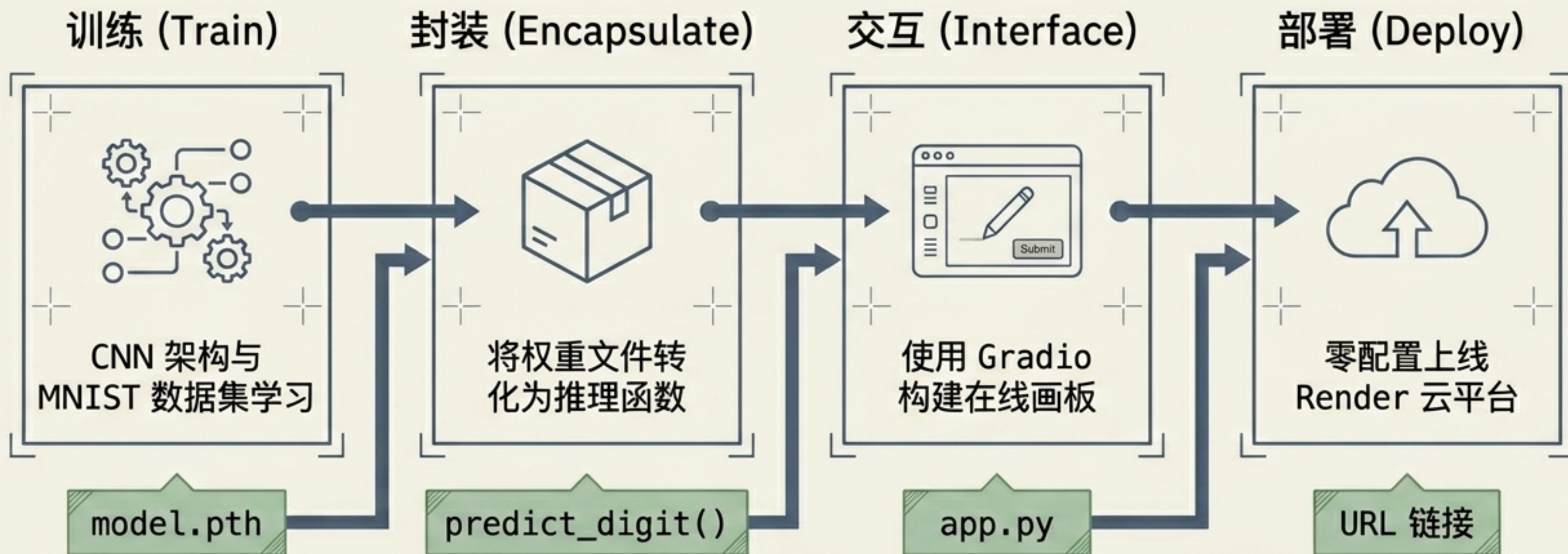
效果显著提升：让学生感受到“自己开发了一个真正的 AI 软件”。

砍掉工程复杂度：专注 AI 核心逻辑

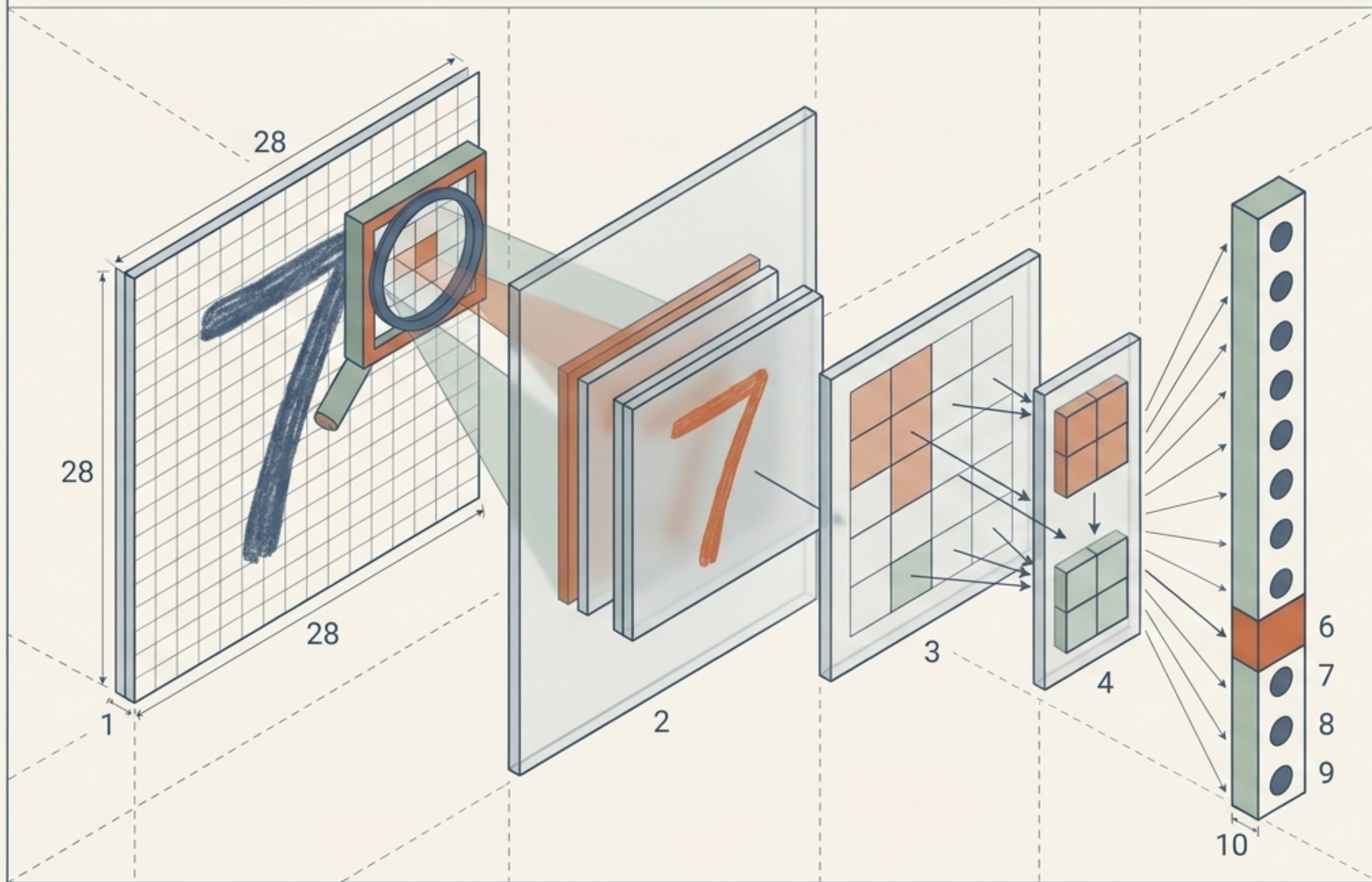
	 传统全栈架构	 最小教学栈
前端框架	Vue / React / HTML+CSS (高学习门槛)	 Gradio (零前端知识需求)
后端与接口	Flask / FastAPI + REST API (需要处理路由、跨域)	 Gradio (内置 Python 原生调用)
交互方式	繁琐的“上传图片”按钮 	 在线画板 Sketchpad (实时绘制识别)
部署环境	AWS / 阿里云 ECS + Nginx (复杂的 Linux 配置) 	 Render Cloud (一键无服务器托管) 

完全砍掉一层架构复杂度，不写任何前后端胶水代码，这是最极致的教学最小闭环。

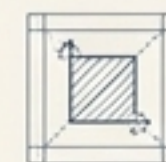
教学最小闭环： 四步构建全栈应用



第一步 / 构造大脑：CNN 网络特征提取



核心优势



空间不变性：无论数字写在画板的哪个位置都能识别。

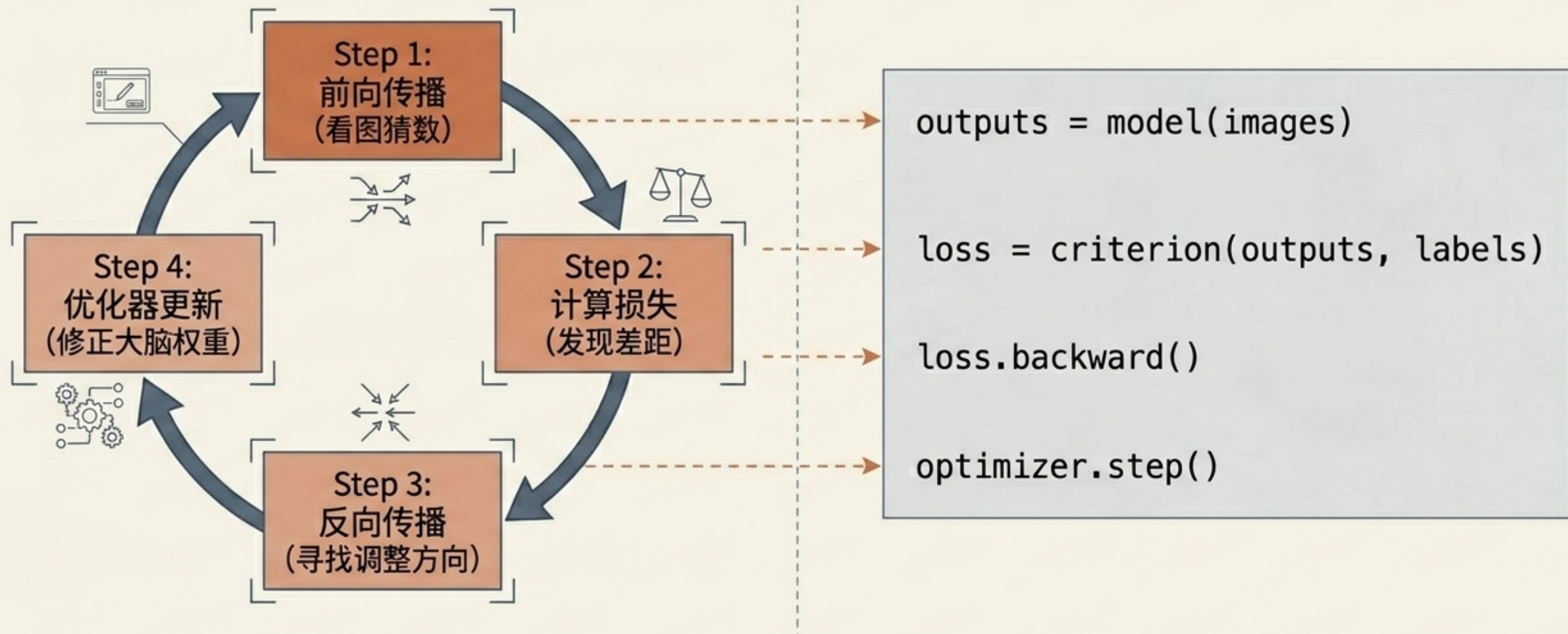


边缘检测：自动提取笔画的线条特征。



参数高效：相比全连接层，大大减少计算负担。

第一步 / 训练循环：让模型学会看懂数字



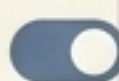
训练产物：导出 `model.pth` 文件（模型权重），这是应用的唯一核心资产。

第二步 / 模型封装：建立推理黑盒

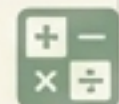
```
def predict_digit(image):
```



1. 载入 model.pth 权重



2. 模型切换至 eval() 模式



3. 执行推理并应用 Softmax

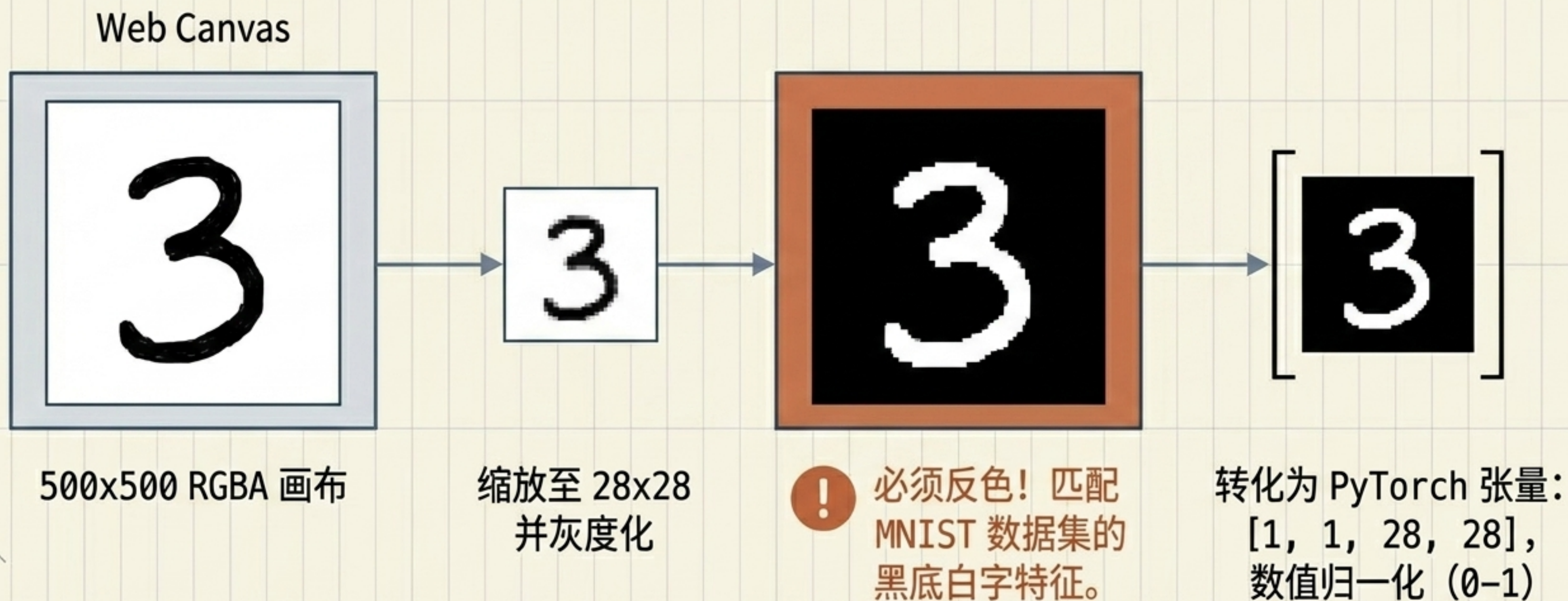


处理后的图片数据

字典格式：{标签: 置信度}

前端画板不需要懂神经网络的原理，它只需要知道：传进一张图，返回一个字典。

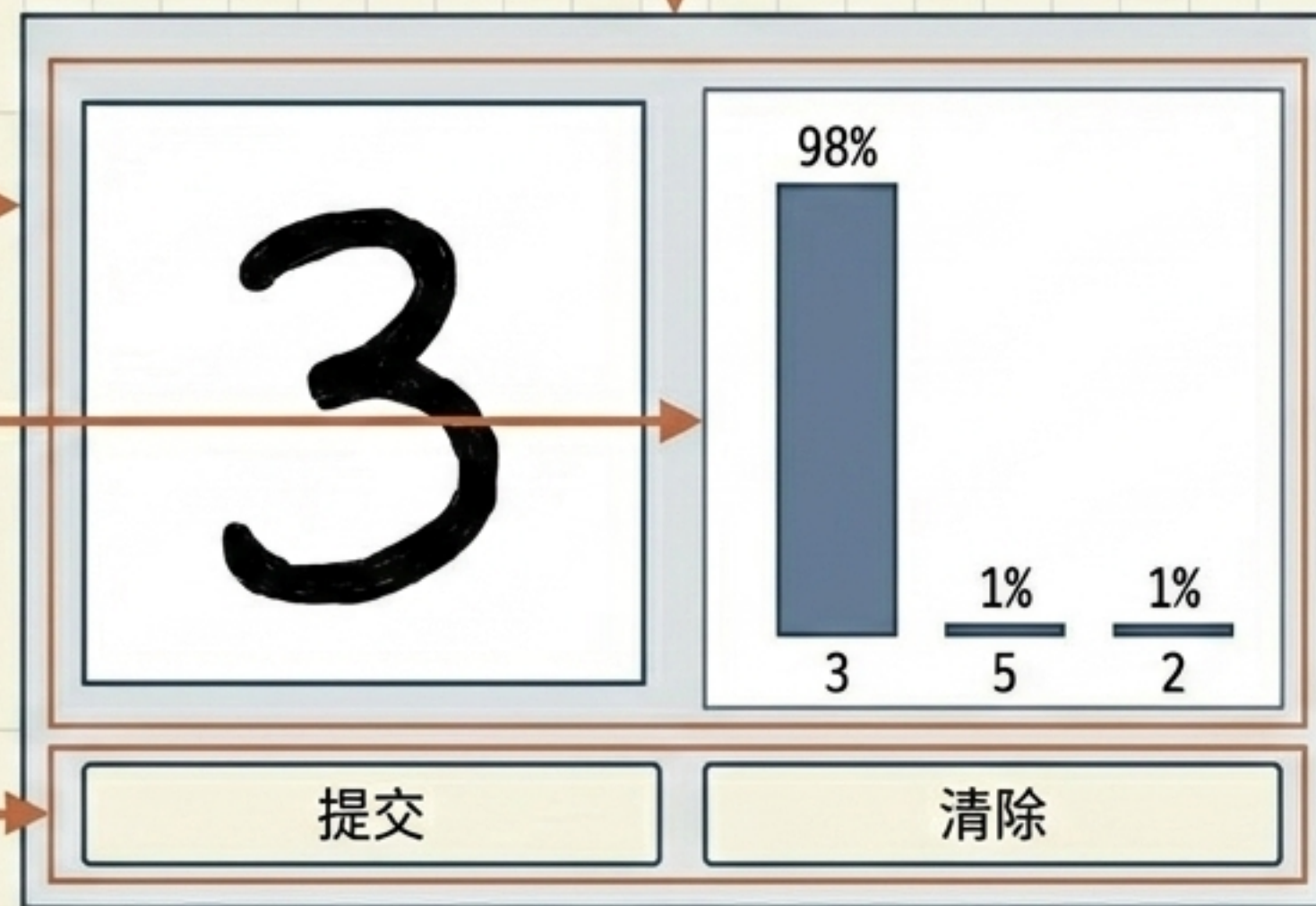
第三步 / 数据桥梁：从网页画板到张量输入



第三步 / 极简前端：三行代码生成交互 UI

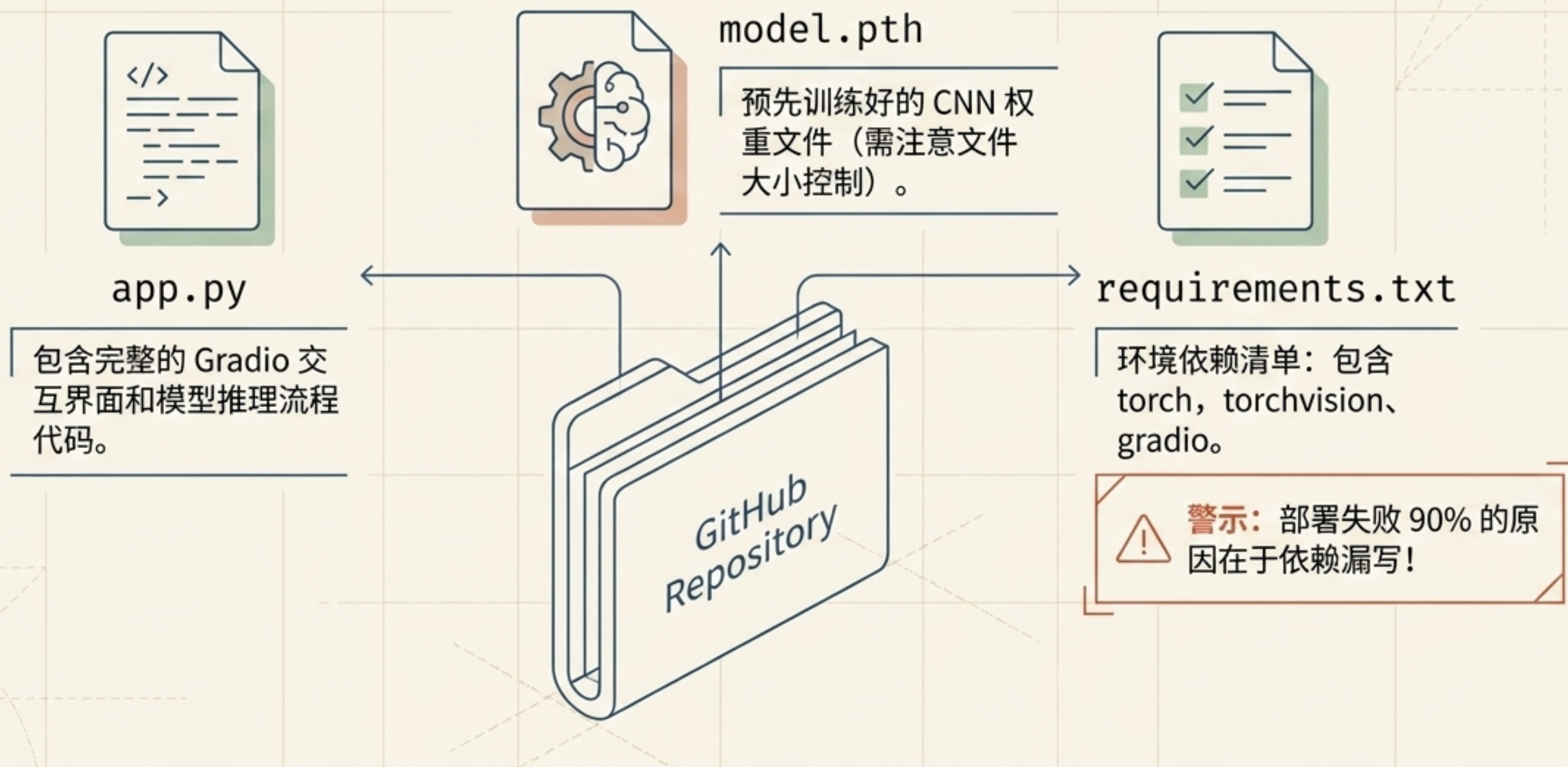
```
inputs = gr.Sketchpad(shape=(28, 28))  
outputs = gr.Label(num_top_classes=3)
```

```
gr.Interface(fn=predict_digit,  
            inputs=inputs, outputs=outputs).  
            .launch()
```



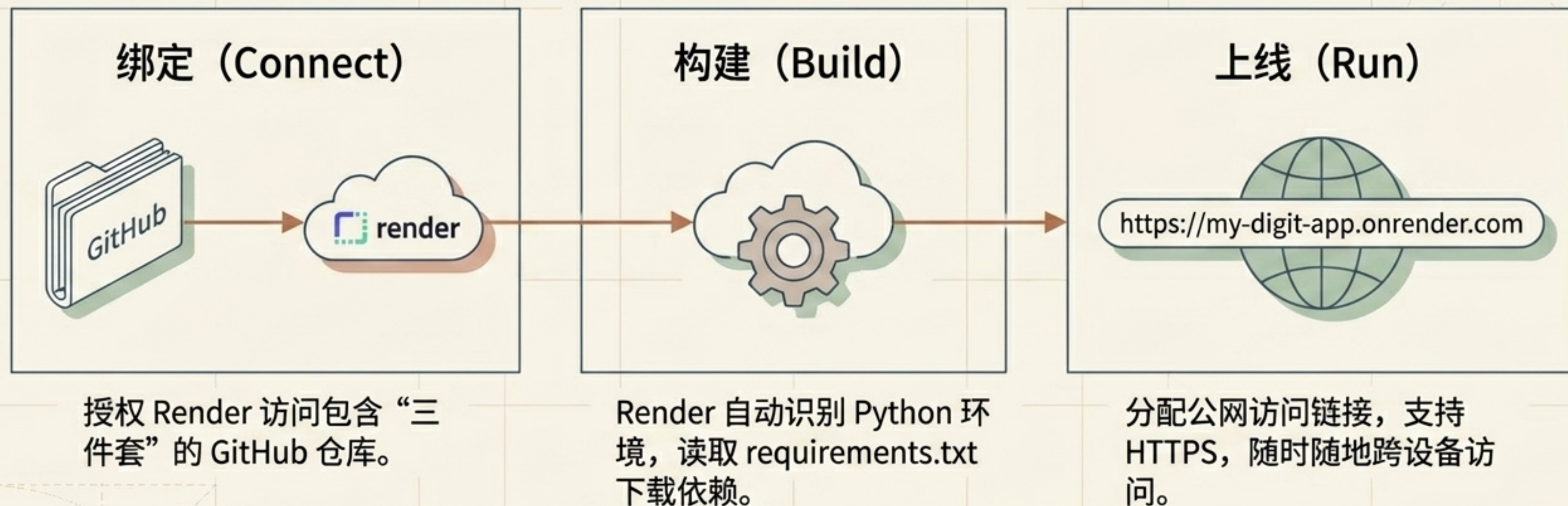
无需编写 HTML/CSS/JS，完全用 Python 声明式定义全栈交互。

第四步 / 打包应用：部署前的“三件套”



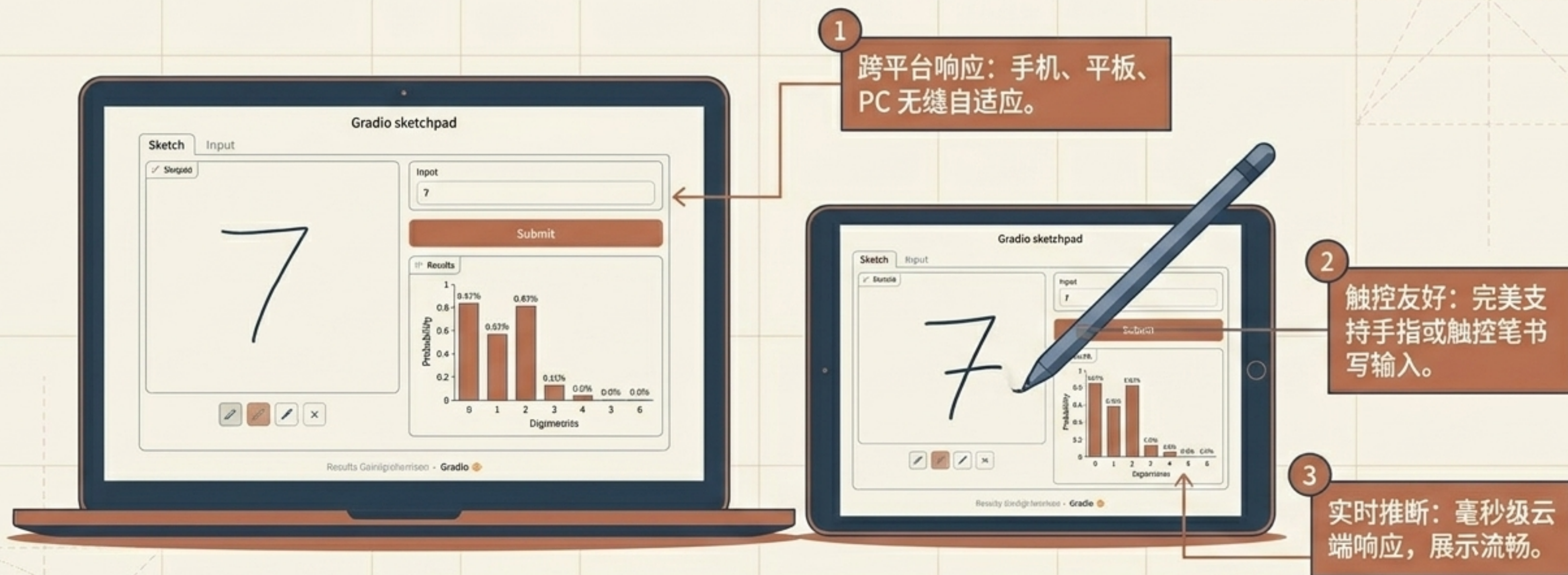
将这三个文件 Push 到 GitHub，即可准备对接云平台。

第四步 / 无服务器部署：Render 平台一键上线



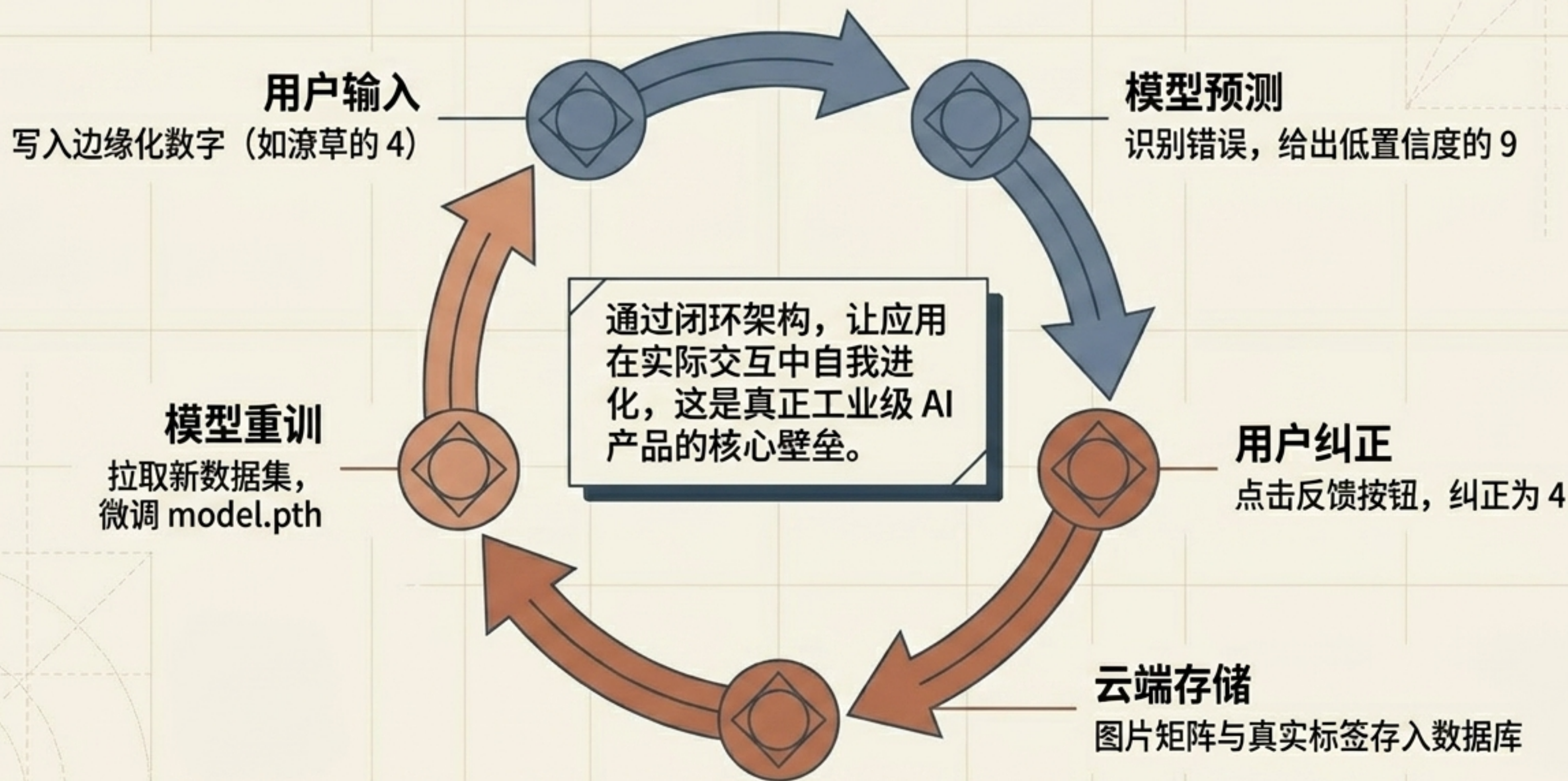
免去了购买服务器、配置 Nginx、处理内网穿透的所有痛苦。

最终交付：从脚本到真实 AI 产品的蜕变



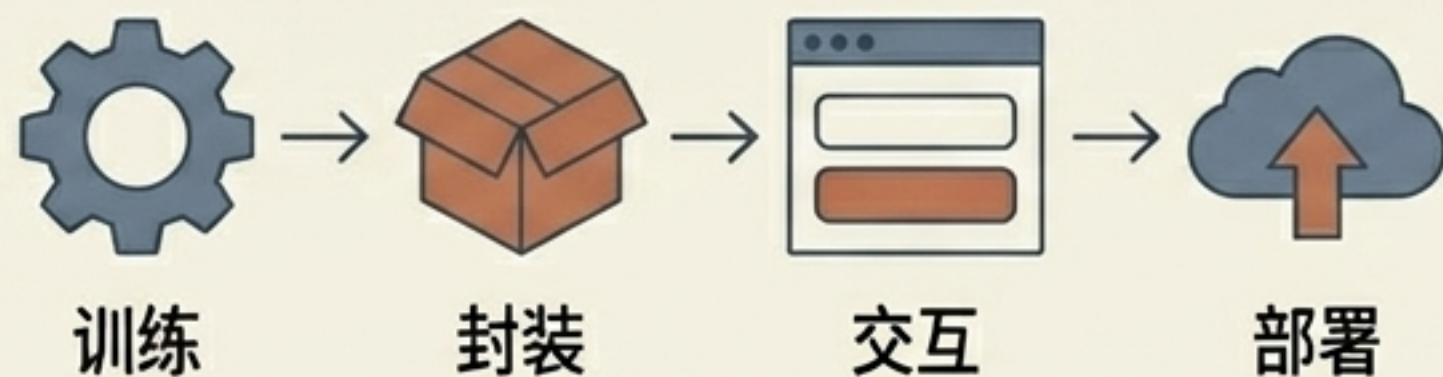
课堂展示效果极佳——当学生亲自写下数字并被瞬间识别时，AI 就不再是黑板上的公式了。

进阶探索：从“单向展示”到“主动学习数据闭环”



架构全景回顾：你的 AI 产品蓝图

架构总结



一套稳定、闭环的工程化蓝图

开发者检查清单

- ☒ 定义 CNN 网络结构并完成训练
- ☒ 导出 model.pth 权重文件
- ☒ 编写 predict_digit(img) 推理函数
- ☒ 3行 Gradio 代码构建在线画板
- ☒ 准备 requirements.txt 并推送到 GitHub
- ☒ 在 Render 完成免运维无服务器部署

掌握这套极简 workflow，即刻将 AI 灵感转化为真实可用产品。